Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-210Б-23

Студент: Домкин П.П.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 10.10.24

Москва, 2024

Постановка задачи

Вариант 9.

В файле записаны команды вида:«число число число<endline>». Дочерний процесс производит деление первого числа команда, на последующие числа в команде, а результат выводит в стандартный поток вывода. Если происходит деление на 0, то тогда дочерний и родительский процесс завершают свою работу. Проверка деления на 0 должна осуществляться на стороне дочернего процесса. Числа имеют тип float. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- pid_t fork(void); – создает дочерний процесс.
- int write(int fd, const void* buffer, int count); – записывает по дескриптору fd count байт из buffer.
- int exec(char* path, const char* argc); – заменяет текущий процесс на процесс path, с аргументами argc;
- pid_t wait(int status) — функция, которая приостанавливает выполнение текущего процесса до тех пор, пока дочерний процесс не завершится.
- void exit(int number); – вызывает нормальное завершение программы с кодом number.
- int shm_open(const char *name, int oflag, mode_t mode); – создает и открывает новый (или открывает уже существующий) объект разделяемой памяти.
- int shm_unlink(const char *name); – удаляется имя объекта разделяемой памяти и, как только все процессы завершили работу с объектом и отменили его распределение, очищают пространство и уничтожают связанную с ним область памяти.
- int ftruncate(int fd, off_t length); – устанавливают длину файла с файловым дескриптором fd в length байт.
- void * mmap(void *start, size_t length, int prot , int flags, int fd, off_t offset); – отражает length байтов, начиная со смещения offset файла (или другого объекта), определенного файловым дескриптором fd, в память, начиная с адреса start.
- int munmap(void *start, size_t length); – удаляет все отражения из заданной области памяти, после чего все ссылки на данную область будут вызывать ошибку "неправильное обращение к памяти".
- sem_t *sem_open(const char *name, int oflag); ИЛИ sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value); – создаёт новый семафор или открывает уже существующий.
- int sem_wait(sem_t *sem); – уменьшает значение семафора на 1. Если семафор в данный момент имеет нулевое значение, то вызов блокируется до тех пор, пока либо не станет возможным выполнить уменьшение.
- int sem_post(sem_t *sem); – увеличивает значение семафора на 1.
- int sem_close(sem_t *sem); – закрывает указанный семафор, освобождая ресурсы, связанные с ним.
- int sem_unlink(const char *name); – удаляет имя семафора из системы. После вызова этой функции другие процессы больше не смогут открыть этот семафор по имени.

В рамках лабораторной работы я реализовал две программы на C: Parent.c и Child.c, которые работают совместно для выполнения вычислений на основе чисел из заданного файла. Программа Parent.c запрашивает у пользователя путь к файлу, создает объект разделяемой памяти для обмена данных с дочерним процессом, использует fork() для его создания и создаёт семафор для синхронизации. Также мы устанавливаем длину объекта разделяемой памяти для

записи туда результатов. Дочерний процесс выполняет программу Child.c. Child.c считывает числа, обрабатывает их, выполняя деление, и записывает результат в разделяемую память. В случае ошибок, таких как неверный ввод или деление на ноль, программа выводит соответствующие сообщения об ошибках. Также родительский процесс ожидает завершение дочернего процесса после вывода результатов.

Код программы

Parent.c

```c
#define SHM_NAME "/shared_memory"
#define SEM_NAME "/sync_semaphore"
#define BUFFER_SIZE 1024
#define NUM_LINES 100

int main() {
    int shm_fd;
    char *shared_mem;
    sem_t *semaphore;

    // Создание разделяемой памяти
    shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (shm_fd == -1) {
        perror("Error creating shared memory");
        exit(EXIT_FAILURE);
    }

    // Изменяет объём памяти
    if (ftruncate(shm_fd, BUFFER_SIZE * NUM_LINES) == -1) {
        perror("Error resizing shared memory");
        exit(EXIT_FAILURE);
    }

    // Отображает BUFFER_SIZE * NUM_LINES байтов в память
    shared_mem = mmap(NULL, BUFFER_SIZE * NUM_LINES, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
    if (shared_mem == MAP_FAILED) {
        perror("Error mapping shared memory");
        exit(EXIT_FAILURE);
    }
```

```c
    // Создание семафора
    semaphore = sem_open(SEM_NAME, O_CREAT, 0666, 0);
    if (semaphore == SEM_FAILED) {
        perror("Error creating semaphore");
        exit(EXIT_FAILURE);
    }

    char filename[BUFFER_SIZE];
    write(STDOUT_FILENO, "Enter filename path: ", 22);
    if (fgets(filename, sizeof(filename), stdin) == NULL) {
        fprintf(stderr, "Error reading filename.\n");
        exit(EXIT_FAILURE);
    }

    size_t len = strlen(filename);
    if (len > 0 && filename[len - 1] == '\n') {
        filename[len - 1] = '\0';
    }

    // Запись имени файла в shared memory
    strncpy(shared_mem, filename, BUFFER_SIZE);

    pid_t child_pid = fork();
    if (child_pid == -1) {
        perror("Error creating child process");
        exit(EXIT_FAILURE);
    }

    if (child_pid == 0) {
        execl("./Child", "./Child", NULL);
        perror("Error executing child process");
        exit(EXIT_FAILURE);
    } else {
        sem_post(semaphore); // Сигнализируем дочернему процессу

        wait(NULL);
```

```c
        // Чтение результата из shared memory
        for (int i = 0; i < NUM_LINES; i++) {
            if (strlen(shared_mem + i * BUFFER_SIZE) > 0) {
                write(STDOUT_FILENO, shared_mem + i * BUFFER_SIZE, strlen(shared_mem + i * BUFFER_SIZE));
            }
        }

        // Удаление shared memory и семафора
        munmap(shared_mem, BUFFER_SIZE * NUM_LINES);
        shm_unlink(SHM_NAME);
        sem_close(semaphore);
        sem_unlink(SEM_NAME);

        exit(EXIT_SUCCESS);
    }
```

Child.c

```c
#define SHM_NAME "/shared_memory"
#define SEM_NAME "/sync_semaphore"
#define BUFFER_SIZE 1024
#define NUM_LINES 100

void HandleError(const char *message) {
    write(STDERR_FILENO, message, strlen(message));
    exit(EXIT_FAILURE);
}

int main() {
    int shm_fd;
    char *shared_mem;
    sem_t *semaphore;

    // Подключение к существующей разделяемой памяти
    shm_fd = shm_open(SHM_NAME, O_RDWR, 0666);
    if (shm_fd == -1) {
        HandleError("Error connecting to shared memory.\n");
    }

    shared_mem = mmap(NULL, BUFFER_SIZE * NUM_LINES, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
    if (shared_mem == MAP_FAILED) {
        HandleError("Error mapping shared memory.\n");
    }

    // Подключение к существующему семафору
    semaphore = sem_open(SEM_NAME, 0);
    if (semaphore == SEM_FAILED) {
        HandleError("Error connecting to semaphore.\n");
    }
```

```c
// Подключение к существующему семафору
semaphore = sem_open(SEM_NAME, 0);
if (semaphore == SEM_FAILED) {
    HandleError("Error connecting to semaphore.\n");
}

// Ожидаем сигнала от родителя
sem_wait(semaphore);

char filename[BUFFER_SIZE];
strncpy(filename, shared_mem, BUFFER_SIZE);

int file = open(filename, O_RDONLY);
if (file == -1) {
    strncpy(shared_mem, "Error: Unable to open file.\n", BUFFER_SIZE);
    sem_post(semaphore);
    exit(EXIT_FAILURE);
}

char buffer[BUFFER_SIZE];
ssize_t bytesRead;
float num_first, num_next;
char *current;
int line_number = 0;

while ((bytesRead = read(file, buffer, BUFFER_SIZE)) > 0) {
    current = buffer;

    while (current < buffer + bytesRead) {
        while (*current == ' ' || *current == '\t') current++;
        if (*current == '\n') {
            current++;
            continue;
        }
```

```c
        char *endptr;
        num_first = strtof(current, &endptr);
        if (endptr == current) {
            strncpy(shared_mem + line_number * BUFFER_SIZE, "Error: Non-numeric value encountered.\n", BUFFER_SIZE);
        }
        current = endptr;

        while (current < buffer + bytesRead && *current != '\n') {
            while (*current == ' ' || *current == '\t') current++;
            if (*current == '\n') break;

            num_next = strtof(current, &endptr);
            if (endptr == current || num_next == 0.0) {
                strncpy(shared_mem + line_number * BUFFER_SIZE, "Error: Division by zero or invalid input.\n", BUFFER_SIZE);
                sem_post(semaphore);
                exit(EXIT_FAILURE);
            }
            num_first /= num_next;
            current = endptr;
        }

        char result[BUFFER_SIZE];
        int result_len = snprintf(result, BUFFER_SIZE, "Result: %.6f\n", num_first);
        strncpy(shared_mem + line_number * BUFFER_SIZE, result, result_len + 1);
        line_number++;

        if (line_number >= NUM_LINES) break;
    }
}

if (bytesRead == -1) {
    strncpy(shared_mem, "Error reading file.\n", BUFFER_SIZE);
    sem_post(semaphore);
    exit(EXIT_FAILURE);
}
```

```c
    close(file);
    sem_post(semaphore);
    exit(EXIT_SUCCESS);
}
```

Протокол работы программы

Тестирование:

```
5.0 4                                   1
10 2 5          6 8 2
2 7
                2 b             c
125 5 0
```

```
pablo@Ardor-Pavel:~/Main/3,4 SEM/LabOs1/LabOS3$ gcc src/Parent.c -o Parent
pablo@Ardor-Pavel:~/Main/3,4 SEM/LabOs1/LabOS3$ gcc src/Child.c -o Child
pablo@Ardor-Pavel:~/Main/3,4 SEM/LabOs1/LabOS3$ ./Parent
Enter filename path: src/test.txt
Result: 1.250000
Result: 0.010417
Result: 0.285714
Error: Division by zero or invalid input.
pablo@Ardor-Pavel:~/Main/3,4 SEM/LabOs1/LabOS3$
```

Strace:

```
strace ./Parent

execve("./Parent", ["./Parent"], 0x7ffc04672ae0 /* 36 vars */) = 0

brk(NULL)                               = 0x55ee26c0f000

arch_prctl(0x3001 /* ARCH_??? */, 0x7ffc7ddf0b90) = -1 EINVAL (Invalid argument)

mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f384f218000

access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=18823, ...}, AT_EMPTY_PATH) = 0

mmap(NULL, 18823, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f384f213000

close(3)                                = 0

openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0P\237\2\0\0\0\0\0"..., 832) = 832

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784

pread64(3, "\4\0\0\0 \0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0\0"..., 48, 848) = 48

pread64(3, "\4\0\0\0\24\0\0\0\3\0\0\0GNU\0I\17\357\204\3$\f\221\2039x\324\224\323\236S"..., 68, 896) = 68

newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=2220400, ...}, AT_EMPTY_PATH) = 0

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784

mmap(NULL, 2264656, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f384efea000

mprotect(0x7f384f012000, 2023424, PROT_NONE) = 0

mmap(0x7f384f012000, 1658880, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7f384f012000
```

mmap(0x7f384f1a7000, 360448, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1bd000) = 0x7f384f1a7000

mmap(0x7f384f200000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x215000) = 0x7f384f200000

mmap(0x7f384f206000, 52816, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f384f206000

close(3) = 0

mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f384efe7000

arch_prctl(ARCH_SET_FS, 0x7f384efe7740) = 0

set_tid_address(0x7f384efe7a10) = 45030

set_robust_list(0x7f384efe7a20, 24) = 0

rseq(0x7f384efe80e0, 0x20, 0, 0x53053053) = 0

mprotect(0x7f384f200000, 16384, PROT_READ) = 0

mprotect(0x55ee06980000, 4096, PROT_READ) = 0

mprotect(0x7f384f252000, 8192, PROT_READ) = 0

prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0

munmap(0x7f384f213000, 18823) = 0

openat(AT_FDCWD, "/dev/shm/shared_memory", O_RDWR|O_CREAT|O_NOFOLLOW|O_CLOEXEC, 0666) = 3

ftruncate(3, 102400) = 0

mmap(NULL, 102400, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) = 0x7f384efce000

openat(AT_FDCWD, "/dev/shm/sem.sync_semaphore", O_RDWR|O_NOFOLLOW) = -1 ENOENT (No such file or directory)

getrandom("\xf9\x55\x08\x2d\x08\xcb\xa5\xf0", 8, GRND_NONBLOCK) = 8

newfstatat(AT_FDCWD, "/dev/shm/sem.lvw2jl", 0x7ffc7ddf04a0, AT_SYMLINK_NOFOLLOW) = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/dev/shm/sem.lvw2jl", O_RDWR|O_CREAT|O_EXCL, 0666) = 4

write(4, "\0\0\0\0\0\0\0\0\200\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0", 32) = 32

mmap(NULL, 32, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0) = 0x7f384f251000

link("/dev/shm/sem.lvw2jl", "/dev/shm/sem.sync_semaphore") = 0

newfstatat(4, "", {st_mode=S_IFREG|0644, st_size=32, ...}, AT_EMPTY_PATH) = 0

getrandom("\xbd\xb8\x32\xe8\x19\xaf\xed\xd5", 8, GRND_NONBLOCK) = 8

brk(NULL) = 0x55ee26c0f000

brk(0x55ee26c30000) = 0x55ee26c30000

unlink("/dev/shm/sem.lvw2jl") = 0

close(4) = 0

write(1, "Enter filename path: \0", 22Enter filename path:) = 22

newfstatat(0, "", {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x6), ...}, AT_EMPTY_PATH) = 0

read(0, src/test.txt

"src/test.txt\n", 1024) = 13

clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x7f384efe7a10) = 45105

wait4(-1, NULL, 0, NULL) = 45105

--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=45105, si_uid=1000, si_status=1, si_utime=0, si_stime=0} ---

write(1, "Result: 1.250000\n", 17Result: 1.250000

) = 17

write(1, "Result: 0.010417\n", 17Result: 0.010417

) = 17

write(1, "Result: 0.285714\n", 17Result: 0.285714

) = 17

write(1, "Error: Division by zero or inval"..., 42Error: Division by zero or invalid input.

) = 42

munmap(0x7f384efce000, 102400) = 0

unlink("/dev/shm/shared_memory") = 0

munmap(0x7f384f251000, 32) = 0

unlink("/dev/shm/sem.sync_semaphore") = 0

exit_group(0) **= ?**

+++ exited with 0 +++

Вывод

В процессе выполнения этой лабораторной работы я освоил работу с новыми системными вызовами в языке C, предназначенными для взаимодействия с семафорами и разделяемой памятью. Я научился обмениваться данными через разделяемую память и управлять доступом с помощью семафоров. Вначале работы возникла путаница с семафорами из-за чего процессы не синхронизировались корректно, однако проблема вскоре решилась. В остальных моментах, проблем с выполнением лабораторной не возникло.